Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tarea - TIA-04 (Unidad 2 - Tarea 2)

Implementación de la estructura completa de una base de datos (DDL) en tres (3) SGBDs diferentes

MIEMBROS DEL EQUIPO:

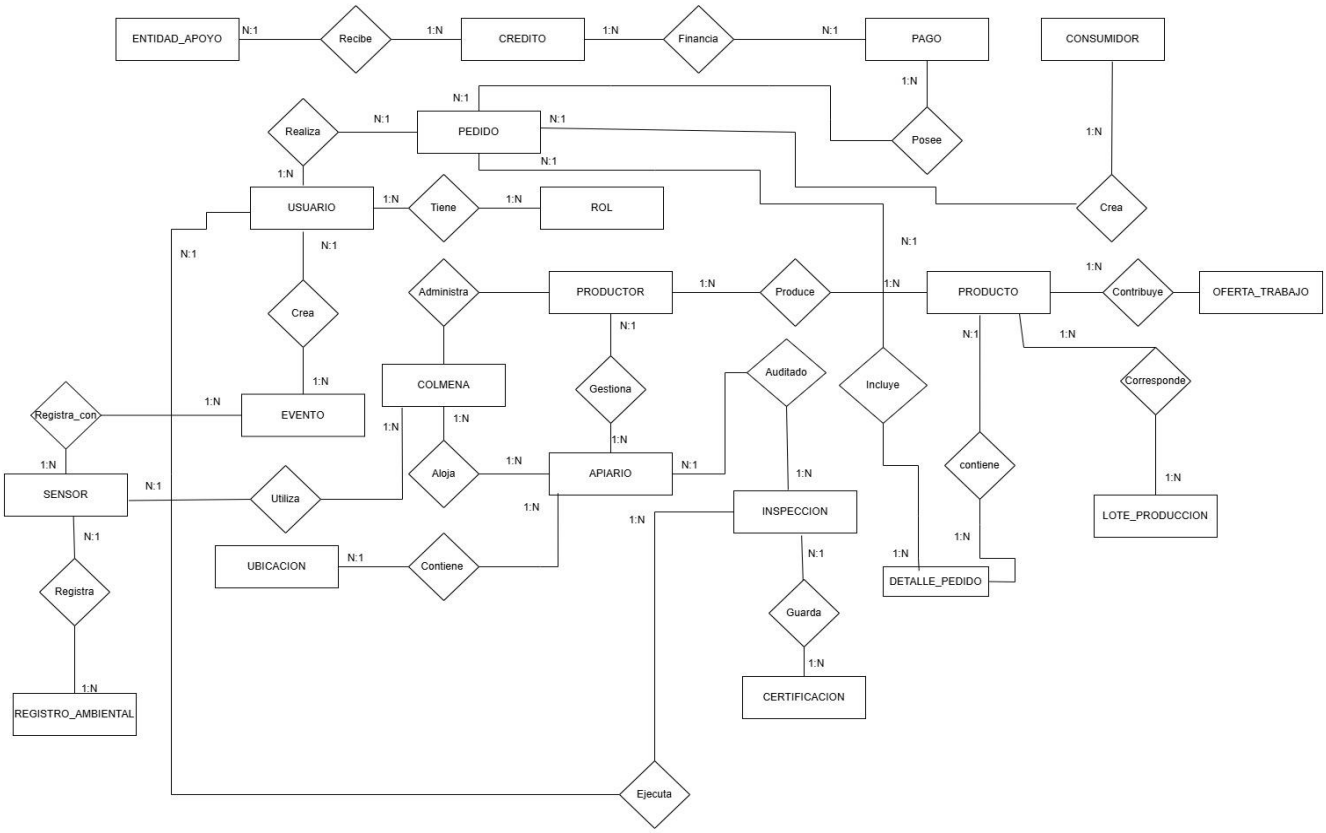
- Líder: Jorge Juan Saldarriaga.
- Miembro: José Julián Doria Ricardo, Nicolás Duque Serna

Informe con resultados

Equipo "4"

(TIA-4: Unidad 2 - Tarea 2)

1.- Diagrama Entidad-Relación de Chen (corregido)



2.- Tabla Comparativa de tipos de Datos

Tipos de Datos - Tabla Comparativa					
#	Tipo de Dato Genérico	Descripción	En PosgreSQL	En MySQL	En MS SQL Server
			18.1	8.0.39	22.5.0
1	INCREMENTABLE	Entero autogenerado	SERIAL / BIGSERIAL	INT AUTO_INCREMENT	INT IDENTITY (1,1)
2	UUID	Identificador único universal	UUID	CHAR(36) / BINARY(16)	UNIQUEIDENTIFIER
3	ENTERO	Número sin decimales	INTEGER	INT	INT
4	DECIMAL	Número con decimales	NUMERIC(p,s)	DECIMAL(p,s)	DECIMAL(p,s)
5	LÓGICO	Verdadero/Falso	BOOLEAN	BOOLEAN / TINYINT(1)	BIT
6	TEXTO	Texto largo	TEXT	TEXT	VARCHAR (MAX)
7	CARACTER	Texto corto fijo	CHAR(n)	CHAR(n)	CHAR(n)
8	FECHA	Solo fecha	DATE	DATE	DATE
9	FECHA_HORA	Fecha y hora	TIMESTAMP	DATETIME	DATETIME
10	HORA	Solo hora	TIME	TIME	TIME
11	URL	Direccion web	TEXT / VARCHAR	VARCHAR(255)	VARCHAR(255)
12	ENUM	Valores predefinidos	ENUM (o CHECK)	ENUM	VARCHAR + CHECK
13	TEXTO_CORTO	Texto limitado	VARCHAR(n)	VARCHAR(n)	VARCHAR(n)
14	BINARIO	Datos binarios	BYTEA	BLOB	VARBINARY(MAX)
15	JSON	Datos JSON	JSON	JSON	NVARCHAR(MAX)
16	JSONB	JSON optimizado	JSONB	No existe	No existe
17	IMAGEN	Imagen	BYTEA	BLOB	VARBINARY(MAX)
18	AUDIO	Audio	BYTEA	BLOB	VARBINARY(MAX)

3.- Inventario de Tablas del Diccionario de Datos Genérico (corregido)

INVENTARIO DE TABLAS Este inventario es el mismo para los 3 SGBDs: PostgreSQL, MySQL y MS SQL Server. Todos los diccionarios deben tomar la misma cantidad y los mismos nombres de esta tablas			
#	Tabla	Descripción Tabla	Observaciones
1	usuario	Almacena los usuarios del sistema	
2	rol	Define los roles disponibles en el sistema	
3	producto	Catálogo de productos apícolas disponibles	
4	productor	Registro de productores apícolas	
5	ubicacion	Coordenadas geográficas de los apiarios	
6	entidad_apoyo	Entidades financiadoras o de apoyo al productor	
7	pedido	Pedidos realizados por los usuarios	
8	pago	Pagos asociados a los pedidos	
9	apiario	Apiarios registrados en el sistema	
10	colmena	Colmenas dentro de los apiarios	
11	inspeccion	Inspecciones realizadas a las colmenas	
12	certificacion	Certificaciones obtenidas tras inspecciones	
13	sensor	Sensores instalados en las colmenas	
14	registro_ambiental	Registros de datos ambientales captados por sensores	
15	evento	Eventos registrados en el sistema	
16	credito	Créditos otorgados a productores por entidades de apoyo	
17	detalle_pedido	Detalle de los productos incluidos en cada pedido	
18	usuario_rol	Tabla intermedia que asocia usuarios con roles	
19	productor_producto	Tabla intermedia que asocia productores con productos	
20	sensor_evento	Tabla intermedia que asocia sensores con eventos	

4. Diccionario de Datos y Scripts de PostgreSQL

4.1- Diccionario de Datos Físico (SGBD PostgreSQL)

PostgreSQL - Tablas en Detalle

1	Tabla	usuario	Descripción Tabla	Almacena los usuarios del sistema					Versión
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	Identificador único del usuario	SERIAL		NO	S			PK
2	nombre	Nombre del usuario	VARCHAR	20	NO				

NOTA: En las columnas PK, FK y UK debe colocar "S" solamente cuando aplique. En la columna "Nulo" coloque "No" en caso de que el campo debe estar valorizado

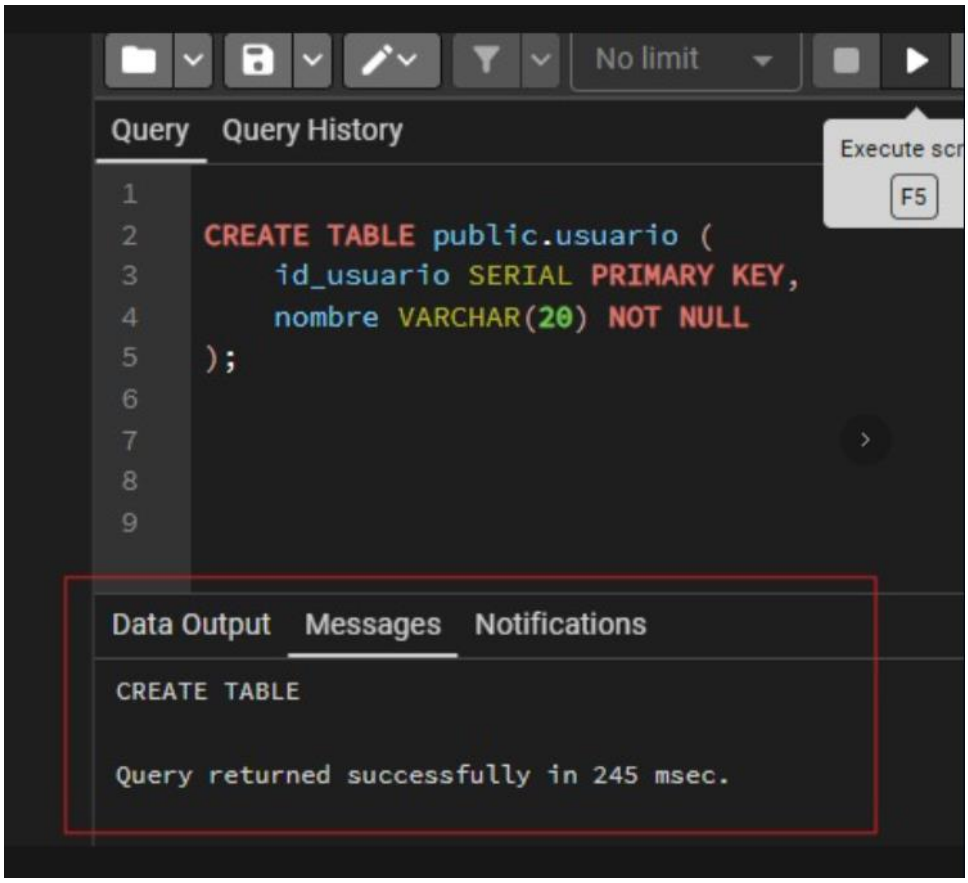
2	Tabla	rol	Descripción Tabla	Define los roles disponibles en el sistema					Versión
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_rol	Identificador único del rol	SERIAL		NO	S			PK
2	nombre	Nombre del rol	VARCHAR	20	NO			S	UNIQUE

NOTA: En las columnas PK, FK y UK debe colocar "S" solamente cuando aplique. En la columna "Nulo" coloque "No" en caso de que el campo debe estar valorizado

3	Tabla	producto	Descripción Tabla	Catálogo de productos apícolas disponibles					Versión
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_producto	Identificador único del producto	SERIAL		NO	S			PK
2	nombre	Nombre del producto	VARCHAR	25	NO				
3	precio	Precio del producto	NUMERIC(10,2)		NO				CHECK precio>=0

NOTA: En las columnas PK, FK y UK debe colocar "S" solamente cuando aplique. En la columna "Nulo" coloque "No" en caso de que el campo debe estar valorizado

4.2.- Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos



4.3..- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos ESTRUCTURADOS

```sql
ALTER TABLE productor
ADD COLUMN telefono VARCHAR(20);

ALTER TABLE productor
ALTER COLUMN telefono TYPE VARCHAR(25);


CREATE TABLE control_produccion (
    id_control SERIAL PRIMARY KEY,
    descripcion TEXT,
    cantidad INTEGER,
    estado VARCHAR(15)
);

ALTER TABLE control_produccion
DROP COLUMN estado;

ALTER TABLE control_produccion
RENAME TO gestion_produccion;

ALTER TABLE gestion_produccion
ADD CONSTRAINT unique_descripcion UNIQUE (descripcion);

ALTER TABLE gestion_produccion
ADD COLUMN fecha_inicio DATE,
ADD COLUMN fecha_fin DATE;

ALTER TABLE gestion_produccion
ADD CONSTRAINT check_fechas CHECK (fecha_fin >= fecha_inicio);

ALTER TABLE gestion_produccion
ADD COLUMN stock INTEGER;

ALTER TABLE gestion_produccion
ADD CONSTRAINT check_stock CHECK (stock >= 0);

ALTER TABLE gestion_produccion
ALTER COLUMN descripcion TYPE VARCHAR(200); SELECT * FROM gestion_produccion;

ALTER TABLE gestion_produccion
ADD CONSTRAINT check_cantidad CHECK (cantidad BETWEEN 1 AND 1000);
```

4.4..- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos para datos

```
TRUNCATE TABLE gestion_produccion;


ALTER TABLE gestion_produccion
ADD COLUMN datos_ambientales JSONB;

INSERT INTO gestion_produccion (descripcion, cantidad, stock, fecha_inicio, datos_ambientales)
VALUES
(
        'Produccion 1',
        50,
        20,
        '2026-03-20',
     '{
        "temperatura": 28.5,
        "humedad": 70,
        "estado": "estable"
    }'
),
(
        'Produccion 2',
        80,
        30,
        '2026-03-21',
        '{
        "temperatura": 30,
        "humedad": 65,
        "estado": "alerta"
    }'
);

SELECT
        descripcion,
        datos_ambientales->>'temperatura' AS temperatura
FROM    gestion_produccion;

ALTER TABLE productor
ADD COLUMN info_extra JSONB;
```

5. Diccionario de Datos y Scripts de MySQL

5.1- Diccionario de Datos Físico (SGBD MySql)

MySQL - Tablas en Detalle

1	Tabla	usuario	Descripción Tabla	Almacena los usuarios del sistema					Versión
#	Campo	Descripción Campo	Tipo Dato - MySQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	Identificador único del usuario	INT AUTO_INCREMENT		NO	S			PK
2	nombre	Nombre del usuario	VARCHAR	20	NO				

NOTA: En las columnas PK, FK y UK debe colocar "S" solamente cuando aplique. En la columna "Nulo" coloque "No" en caso de que el campo debe estar valorizado

2	Tabla	rol	Descripción Tabla	Define los roles disponibles en el sistema					Versión
#	Campo	Descripción Campo	Tipo Dato - MySQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_rol	Identificador único del rol	INT AUTO_INCREMENT		NO	S			PK
2	nombre	Nombre del rol	VARCHAR	20	NO			S	UNIQUE

NOTA: En las columnas PK, FK y UK debe colocar "S" solamente cuando aplique. En la columna "Nulo" coloque "No" en caso de que el campo debe estar valorizado

3	Tabla	producto	Descripción Tabla	Catálogo de productos apícolas disponibles					Versión
#	Campo	Descripción Campo	Tipo Dato - MySQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_producto	Identificador único del producto	INT AUTO_INCREMENT		NO	S			PK
2	nombre	Nombre del producto	VARCHAR	25	NO				
3	precio	Precio del producto	DECIMAL(10,2)		NO				CHECK precio>=0

NOTA: En las columnas PK, FK y UK debe colocar "S" solamente cuando aplique. En la columna "Nulo" coloque "No" en caso de que el campo debe estar valorizado

5.2.- Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos (MySQL)

6. Diccionario de Datos y Scripts de MS SQL Server

6.1- Diccionario de Datos Físico (SGBD MS SQL Server)

MS SQL Server - Tablas en Detalle

1	Tabla	usuario	Descripción Tabla	Almacena los usuarios del sistema					Versión
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	Identificador único del usuario	INT IDENTITY(1,1)		NO	S			PK
2	nombre	Nombre del usuario	VARCHAR	20	NO				

NOTA: En las columnas PK, FK y UK debe colocar "S" solamente cuando aplique. En la columna "Nulo" coloque "No" en caso de que el campo debe estar valorizado

2	Tabla	rol	Descripción Tabla	Define los roles disponibles en el sistema					Versión
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_rol	Identificador único del rol	INT IDENTITY(1,1)		NO	S			PK
2	nombre	Nombre del rol	VARCHAR	20	NO			S	UNIQUE

NOTA: En las columnas PK, FK y UK debe colocar "S" solamente cuando aplique. En la columna "Nulo" coloque "No" en caso de que el campo debe estar valorizado

3	Tabla	producto	Descripción Tabla	Catálogo de productos apícolas disponibles					Versión
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_producto	Identificador único del producto	INT IDENTITY(1,1)		NO	S			PK
2	nombre	Nombre del producto	VARCHAR	25	NO				
3	precio	Precio del producto	DECIMAL(10,2)		NO				CHECK precio>=0

NOTA: En las columnas PK, FK y UK debe colocar "S" solamente cuando aplique. En la columna "Nulo" coloque "No" en caso de que el campo debe estar valorizado

6.2.- Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos (MS SQL Server)

```sql
CREATE TABLE usuario (
    id_usuario INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    nombre VARCHAR(20) NOT NULL
);

CREATE TABLE rol (
    id_rol INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    nombre VARCHAR(20) NOT NULL UNIQUE
);

CREATE TABLE producto (
    id_producto INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    nombre VARCHAR(25) NOT NULL,
    precio DECIMAL(10,2) NOT NULL CHECK (precio >= 0)
);

CREATE TABLE productor (
    id_productor INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    nombre VARCHAR(20) NOT NULL
);
```

100 % — No se encontraron problemas.

Mensajes

```
Los comandos se han completado correctamente.

Hora de finalización: 2026-04-16T20:48:21.3047589-05:00
```

7.- Análisis de resultados

En nuestra conclusión sobre el trabajo realizado, podemos decir que fue una experiencia bastante completa, ya que no solo diseñamos la base de datos, sino que también la implementamos en tres gestores diferentes como mysql, sql server y postgresql. esto nos permitió comparar directamente cómo funciona cada uno y entender que, aunque el modelo es el mismo, la implementación cambia en varios aspectos. aunque el modelo conceptual y lógico se mantuvo igual, al llevarlo al modelo físico encontramos diferencias importantes. por ejemplo, en mysql utilizamos enum para el atributo estado de la colmena, lo cual facilita controlar los valores permitidos, mientras que en postgresql y sql server este tipo de dato no se maneja igual, por lo que fue necesario usar restricciones check. esto nos mostró que algunas decisiones pueden afectar la compatibilidad entre sistemas. también vimos diferencias en los tipos de datos. aunque usamos decimal para valores como precios, montos, temperatura y humedad en los tres gestores, el manejo interno puede variar. en cuanto a fechas, mysql utiliza datetime, postgresql trabaja más con timestamp y sql server también usa datetime, pero con ciertas diferencias. esto puede influir en consultas y almacenamiento. otro punto clave fue el autoincremento de las claves primarias. en mysql usamos auto_increment, en sql server identity y en postgresql serial. todos cumplen la misma función, pero la sintaxis cambia, lo que obliga a adaptar los scripts. sobre las restricciones, notamos que postgresql es más estricto en el cumplimiento de reglas como check, mientras que mysql puede ser más flexible dependiendo de la versión, mientras que sql server también maneja bien las restricciones, pero con una sintaxis distinta. en general, este trabajo nos ayudó a entender que diseñar una base de datos no es solo crear tablas, sino también saber adaptarla a diferentes gestores. además, el trabajo en equipo fue importante para organizarnos y avanzar en paralelo. finalmente, concluimos que el modelo está bien estructurado, pero que lograr compatibilidad total entre gestores es un reto y siempre habrá detalles por ajustar.

8.- Reflexiones y conclusiones individuales

conclusión individual – Nicolas Duque: en mi caso, lo que más me dejó este trabajo fue entender que el diseño de una base de datos no es solo hacer tablas, sino pensar bien la estructura desde el inicio. me di cuenta de que definir correctamente las entidades y relaciones en el modelo conceptual es clave para que todo lo demás funcione bien. algo que me llamó la atención fue cómo pequeños detalles, como elegir un tipo de dato o poner una restricción, pueden afectar todo el sistema. en lo académico, esto me ayudó a ver la materia de una forma más aplicada y no solo teórica. a nivel profesional, siento que me da una base importante porque sé que este tipo de cosas se usan mucho en sistemas reales. en el trabajo en equipo, creo que lo más importante fue organizarnos, porque si no, el trabajo se volvía desordenado. también aprendí a confiar en lo que hacía cada compañero. el caso de estudio me pareció útil porque permitió trabajar algo más cercano a una situación real y no solo ejercicios simples.

conclusión individual – Jorge Juan Saldarriaga: para mí, este trabajo fue más que todo un ejercicio práctico para entender cómo se implementa una base de datos en diferentes sistemas. lo que más me aportó fue comparar mysql, sql server y postgresql, ya que ahí fue donde realmente vi que aunque la estructura sea la misma, cada gestor tiene su propia forma de manejar cosas como tipos de datos, restricciones y sintaxis. eso me hizo entender que uno como desarrollador tiene que ser flexible y saber adaptarse. en lo académico, esto me ayudó a reforzar conceptos que ya había visto, pero ahora con más sentido. pensando en lo profesional, creo que es muy útil porque no siempre se trabaja con un solo gestor. sobre el trabajo en equipo, hubo momentos en los que tocó coordinar bastante para que todo quedara bien, pero eso ayudó a mejorar la comunicación. el modelo del caso de estudio fue importante porque permitió ver cómo todo esto se aplica en algo más real.

conclusión individual – Jose Julian Doria: desde mi punto de vista, este trabajo me ayudó principalmente a entender la importancia de los detalles en una base de datos. cosas como las claves foráneas, las restricciones o incluso cómo se nombran las tablas, influyen bastante en el resultado final. antes no le prestaba tanta atención a eso, pero ahora veo que son fundamentales para que todo funcione correctamente. también me pareció interesante trabajar con varios gestores, porque ahí se nota que no todo es copiar y pegar, sino que hay que adaptar el código según el sistema. en mi formación académica, esto me ayudó a tener una visión más completa del tema. a nivel profesional, creo que es un conocimiento necesario. en cuanto al trabajo en equipo, aunque a veces fue complicado, logramos sacar el trabajo adelante y eso también deja aprendizaje. el caso de estudio ayudó a que todo tuviera más sentido y no fuera solo teoría.